

Coursework 1: MA22019, Introduction to Data Science

Analyzing Movie Metrics, Demographics, and Genres

[Download PDF version of these instructions](#)

Coursework cover sheet

- **Set:** 12:00 on 5 March 2026
- **Due:** 17:00 on 13 March 2026
- **Estimated time required:** This coursework should take no more than 10 hours to complete.
- **Value:** This assignment carries 40% of the total marks for the unit.
- **Length:** A strict word count check applies to the final written interpretation section. Limit your total written answers to a maximum of **1,200 words**. Keep your answers concise!
 - **What IS counted:** The prose you write underneath the `Written Interpretation` headers for each task.
 - **What is NOT counted:** Your R code snippets (````r` blocks), the calculated output of your R code (tables, charts), code comments (`#`), YAML headers, instructional HTML comments (`<!-- -->`), and Markdown section headers (`###`). The automated word count script dynamically strips these out before calculating your total.
- **Feedback:** You will receive feedback within a maximum of three semester weeks following the submission deadline. The feedback will consist of your marked work and an overall feedback document commenting on the assessment across the cohort.
- **Getting the coursework:** Navigate to your coursework repository folder on <https://github.bath.ac.uk/ma22019-2026/coursework-01-yourusername>. Substitute `yourusername` with your actual University of Bath username.

- **Git Users:** Navigate to your `MA22019/coursework/` folder (create the `coursework` folder if it doesn't exist yet). Clone your coursework repository into this folder as usual.
- **Non-Git Users:** Click the green **Code** button, and select **Download ZIP**. Extract the ZIP file and move the extracted folder into your `MA22019/coursework/` folder (create the `coursework` folder if it doesn't exist yet).

You can check the [coursework workflow](#) for more details. You will be provided with the following files and folders:

- `coursework_01.qmd` The template file for the coursework and to be filled with your code and answers.
- `data/` The data files for the coursework.
- `coursework_01.Rproj` The file to open the project in RStudio.

Your repository also contains some additional read-only files that you do not need to edit or submit:

- `coursework_01_instructions.html / .pdf` — A local copy of these instructions for your reference.
- `.github/` — An internal workflow folder used by the system. **Do not edit or delete this folder.**

- **Structural Constraints:** In the template file:

- Do not alter the header (the text in between `---`) apart from adding your name.
- Do NOT delete any of the HTML anchors (e.g., `<!-- DO NOT DELETE THIS LINE --->` or `<!-- WRITE YOUR ANSWER BELOW THIS LINE --->`).
- Do not remove the code chunk labels eg `#| label: task-1-code`
- Do NOT delete the final R code chunk at the bottom of the template that calculates your word count.

Your assignment will be processed by a standardised data ingestion pipeline, and breaking the structure will cause your submission to fail the checks.

- **Submission:** Electronically via `github.bath.ac.uk/ma22019-2026/coursework-01-yourusername`. You should submit the following files and folders:

- `coursework_01.qmd`. The updated version with your code and answers.
- the generated `coursework_01_files/` folder (which contains your plots).
- the `data/` folder (intact as provided).

You only need to submit your own work — do not worry about the extra read-only files (`coursework_01_instructions.*`, `.github/`), they can remain in your repository as-is.

Use the following submission workflow.

- **Git Users:** Commit and push regularly (at least three times as per [coursework workflow](#)).
- **Non-Git Users:** You can upload as many times as you like before the deadline. To upload your submission files to your homework repository:
 1. Click on the **Add file (+)** button at the top of your repository page.
 2. Select **Upload files** from the dropdown menu.
 3. Drag and drop your three required items (`coursework_01.qmd`, the `coursework_01_files/` folder, and the `data/` folder) into the upload box. Do NOT upload a ZIP file to Github.
 4. Wait for them to load.
 5. In the “Commit changes” message box below the files, type strongly: “my finalised submission”.
 6. Click the green **Commit changes** button at the bottom (leave “Commit directly to the main branch” selected).
 7. Verify that all four items now appear in your repository file list.
- **Style & Naming Conventions:** Please adhere strictly to the variable and dataset names requested in the tasks. All plots must include clear, descriptive axis labels and titles — avoid raw variable names on axes (e.g., use “User Rating (out of 100)” rather than `user_rating_100`).
- **Conditions:** This is an individual coursework, and you should not discuss it with anyone other than the lecturer or your tutor. It should be completed during your computer lab on 9 or 10 March and in your own time.
- **Support and advice:** You will be able to ask questions during the lectures and during your tutorials. Questions can also be posted on Moodle or the [MA22019 Padlet board](#). You can ask generic statistical, coding or presentation questions relevant to the coursework or the course in general, but not specific questions about how to do the coursework analyses.
- **Late submission of coursework:** If there are valid circumstances preventing you from meeting the deadline, your Director of Studies may grant you an extension to the specified submission date, if it is requested before the deadline. Forms to request an extension are available on SAMIS.
 - If you submit a piece of work after the submission date, and no extension has been granted, the maximum mark possible will be the pass mark.
 - If you submit work more than five working days after the submission date, you will normally receive a mark of 0 (zero), unless you have been granted an extension.
- **Academic integrity statement:** Academic misconduct is defined by the University as “*the use of unfair means in any examination or assessment procedure*”. This includes (but is not limited to) cheating, collusion, plagiarism, fabrication, or falsification. The University’s Quality Assurance Code of Practice, [QA53 Examination and Assessment](#)

[Offences](#), sets out the consequences of committing an offence and the penalties that might be applied.

- **Generative AI: Type B:** “GenAI is permitted as an assistive tool for specific defined processes within the assessment and its use is not mandatory in order to complete the assessment.”
 - GenAI tools may be used to answer any question. Under the University’s Academic Integrity Statement, you ‘must not present content created by generative AI tools as though it were your own’. Any text or code produced by genAI must be checked for correctness and cited. In addition, you must include the mandatory declaration indicating what tools you used and how you used them OR that you have not used genAI. You should be prepared to explain anything in your submission to an examiner if asked to do so.
 - See [Gen AI Assessment Categorisation](#)
 - **Mandatory Action:** You must complete the MS Form Gen-AI Declaration before submitting your coursework: [Link to MS Form](#)
-

Overview

Welcome to Coursework 1! In this assignment, you are working as an analyst for **CineMetrics**, a (fictional) streaming subscription service. The data engineering team has provided you with three raw data frames containing metadata, user information, and a log of every movie watched on the platform. Your goal is to perform data wrangling, exploratory data analysis, and text mining to derive actionable insights about user engagement and content performance.

Empirical Citation Rule: All written interpretations MUST cite at least two computed numerical values from your code output as evidence. General conceptual explanations without empirical citations will receive partial credit at best.

The Data

You have three datasets in your `data/` folder:

1. `titles.csv`: Contains metadata about the movies.
 - `movie_id`: Unique identifier for the movie.
 - `movie_title`: The name of the movie.
 - `release_year`: Year of release.
 - `budget_usd`: Original production budget in USD
 - `genre_tags`: A messy, comma-separated list of micro-genres or thematic tags assigned by users.
 2. `users.csv`: Information about CineMetrics subscribers.
 - `user_id`: Unique user identifier.
 - `subscription_type`: “Basic” or “Premium”.
 - `age`: User’s age.
 3. `history.csv`: The streaming logs.
 - `user_id`: The user who watched.
 - `movie_id`: The movie they watched.
 - `watch_date`: The date it was watched.
 - `watch_duration_mins`: How many minutes were viewed.
 - `engagement_score`: A calculated metric representing user attention.
 - `user_rating_100`: A rating given by the user (out of 100).
 - `review_snippet`: Optional written review left by the user.
-

Part A: Data Wrangling & EDA

Task 1: Data Joining (8 Marks)

If you load the `history` and `titles` data frames, you might notice that some rows were accidentally duplicated due to a server glitch.

First, clean the data frames so that explicit full-row duplicates are removed. Next, create a master dataset named `cinemetrics` by joining the clean `history`, `users`, and `titles` data.

- **Written Interpretation:** In the written template section, explain in one single sentence why it was necessary to remove duplicates in the `history` data frame *before* joining it to the others. Also, what would have happened to the total row count if you hadn’t removed the duplicates in the `titles` data frame prior to the join?

Task 2: Missing Values Handling (8 Marks)

Produce a cleanly formatted 2-column data frame (use `knitr::kable()`) named `missing_summary`. The columns must be named `Variable` and `Missing_Count`, showing the number of missing values per variable for the entirety of the merged `cinematics` dataset. Afterwards, calculate the overall median `user_rating_100` for the dataset.

Formatting Tip: `knitr::kable()` To make your data frames look clean and professional in your final document, you can wrap your small dataframe summaries in `knitr::kable()`. For example:

```
my_summary_df |>
  knitr::kable(caption = "My Clean Data Frame")
```

- **Written Interpretation:** Why is it mathematically necessary to programmatically evaluate missing values using explicitly designed functions rather than relying on the output of generic data overview functions? Which specific data type do these generic overviews typically fail to flag missing values for?

Task 3: Genre Grouping (12 Marks)

The `genre_tags` column contains a messy array of granular categories (e.g., “rom-com”, “Action-Packed”, “sci fi”). We want to standardise the raw `genre_tags` column into a new variable called `primary_genre` instead.

However, the raw data still has too many distinct small categories. Using a **single command** to collapse the `genre_tags` into the `primary_genre` variable with exactly 8 levels. Keep the top 5 most frequent genres exactly as they are. Collapse *every other genre* into a new level called ‘Other’, and combine spelling variations of “Sci-Fi” and “SciFi” into “Sci-Fi”. You must exclusively use the `forcats` package for this manipulation (no `case_when` or `ifelse`).

- **Empirical Verification:** Write a short data pipeline (in the `task-3-verification` chunk) to calculate the average `user_rating_100` and the count of movies (`n()`) grouped by the raw, original `genre_tags` column. Look specifically at the difference between the massive “Romance” tag and the specific “romantic comedy” tag. Display this comparison as a formatted table using `knitr::kable()`.
- **Written Interpretation:** Based on the empirical numbers you just calculated, what specific statistical distortion occurs to the “romantic comedy” ratings when they are forcefully collapsed together into the single “Romance” category?

Task 4: Genre Ratings Data Frame (6 Marks)

Group the data by your newly created `primary_genre`, calculate the average user rating, and display it as a cleanly formatted table (use `knitr::kable()`). Genres should be listed in descending order in terms of average user rating.

- **Empirical Verification:** To investigate the relationship between budget and quality, write a short data pipeline (in the `task-4-verification` chunk) to calculate the average production budget (`budget_usd`) for the `Horror`, `Action`, and `Sci-Fi` genres (displaying the result with `knitr::kable()`).
- **Written Interpretation:** You will likely notice a massive disparity in average ratings between the top-performing genre and the lowest-performing genres on your table. This raises the question as to whether a higher production budget guarantees a higher average user rating. Based *strictly* on these empirical budget numbers and your genre rating table, describe the relationship between production budget and average user rating for these three genres. Do you think creating a massive, generalized `Action` category hides the nuances of specific sub-genres within it? Explain how the poor performance of a small, specific niche film group can be masked when calculating the average rating of a massive, collapsed category.

Task 5: Ratings and Engagement Scores (14 Marks)

Find the top 10 movies with the highest number of total watches.

For *just* these 10 movies, create a visualization using a faceted boxplot showing the distribution of `user_rating_100`.

Visual Requirements:

- You must strictly use a single `mutate` pipeline to re-order the factors of the movie titles so that the movie with the highest **median** rating appears first in the facets.
- Ensure the faceted layout is legible in a single row.
- The x-axis should not have any text or tick marks. The movie titles should instead serve as the strip labels for each facet.
- **Written Interpretation:** Based on the visual footprint of your boxplots, which of the 10 movies contains the single lowest individual user rating (the lowest outlier dot on the y-axis)? What does the size of the box imply about the audience's consensus?
- **Empirical Verification:** To verify your visual interpretation of the box sizes, write a short data pipeline (in the `task-5-verification` chunk) to calculate the exact Interquartile Range of the ratings for each of these 10 movies. Display the results in a cleanly formatted table, sorted by IQR in descending order. Which movie has the highest IQR (largest box), and which has the lowest (smallest box)?

Task 6: Engagement Score and Subscription Type (14 Marks)

We want to analyze if `Premium` users are more engaged than `Basic` users. First, calculate the average `engagement_score` grouped solely by `subscription_type` (2D Analysis).

Next, look additionally at differences across Age (3D Analysis). First, determine the quartiles (25%, 50% and 75%) of the `age` column to discover the exact boundaries of your age groups. Then, create a new `age_tier` variable that categorizes users into these four data-driven brackets. Finally, calculate the average `engagement` grouped by *both* `age_tier` and `subscription_type`.

- **Empirical Verification:** We need to know *who* is actually using these two subscriptions. Write a short data pipeline (in the `task-6-verification` chunk) to calculate the total count of users (`n()`) grouped by both `age_tier` and `subscription_type`. Display this as a wide, cleanly formatted table using `tidyr::pivot_wider()`, so that the age tiers act as the rows and the two subscription types act as the columns.
 - **Written Interpretation:** Look closely at the results from your 2D Analysis (global) compared to your 3D Analysis (segmented by age). Identify the contradictory pattern present when comparing the subscription types across these two tables. What is this mathematical illusion called? Explain *why* this happens using empirical data evidence from the tables (including your calculated user counts) to support your reasoning.
-

Part B: Text Data Analysis

Task 7: Tokenisation & Word Frequency (6 Marks)

Tokenize the `review_snippet` into separate, lowercase words. Remove standard English stop words using the appropriate text-mining package. Create a simple bar chart showing the frequencies of the top 10 most used words across the entire dataset.

- **Written Interpretation:** Looking at your bar chart, what are the two most frequent words (excluding stop words)? Why might these specific words be completely unhelpful if we wanted to understand why an audience rated a movie highly, or what makes a specific genre's reviews unique?

Task 8: Sentiment Analysis (12 Marks)

Perform a sentiment analysis using the "bing" lexicon. Calculate a Net Sentiment score (Positive words minus Negative words) for each review. Then, calculate the **average Net Sentiment** and the **average user_rating_100**, grouped by **primary_genre**. Create a scatter plot comparing Average Sentiment (x-axis) vs Average Rating (y-axis).

- **Written Interpretation:** Examine your scatter plot carefully. Identify any genres that appear as an anomaly when compared to the general trend of the rest of the dataset. Describe where they sit on the plot and explain why this semantic anomaly occurs.

Task 9: Advanced Text Mining - TF-IDF (12 Marks)

Stop words don't catch everything. Often, words like "movie" or "watch" are extremely high-frequency across *all* genres, making them useless for understanding what makes a genre unique. Apply the `bind_tf_idf()` function from `tidytext` to your tokenized dataset. Find the words with the highest TF-IDF scores for each **primary_genre**.

- **Written Interpretation:** Look closely at your TF-IDF data for the **Romance** genre and answer the following three questions:
 1. What is the TF-IDF score for the word "film"?
 2. Based on the TF-IDF mathematical formula covered in the lecture, explain exactly *which part* of the equation caused "film" to receive this specific score in your dataset.
 3. Compare the top 3 highest TF-IDF words for Romance against the top 3 highest *frequency* (`n`) words. Why does TF-IDF give a better, more unique summary of the genre?

Task 10: Executive Recommendation (8 Marks)

You are writing a **one-paragraph executive summary** (max 150 words) for CineMetrics' Head of Content. Based on the evidence you have gathered across Tasks 1–9, make **one specific, data-backed recommendation** that CineMetrics should implement to improve audience satisfaction or platform growth.

Your recommendation must:

- Reference at least **three specific findings** from your earlier tasks (citing the task number and the empirical values).
- Acknowledge at least **one limitation** of the data or analysis that could affect your recommendation.

Good luck, and remember to commit and push frequently if you are using Git!